BÁO CÁO TIẾN ĐỘ TUẦN 3

Nghiên cứu kiến trúc ma trận 2D AES-128, cơ chế sinh khóa, PKCS#7 Padding, chế độ khối CBC và cài đặt mã hóa/giải mã File & ZIP đa năng

Học phần PBL3: Mạng thông tin máy tính

GVHD: TS. Trần Thị Minh Hạnh

SVTH: Nguyễn Thái Hoàng – 106230011

Lê Trương Uyển Nhi – 106230054

Tóm tắt nội dung Tuần 3

Tiếp nối kết quả nghiên cứu lõi thuật toán ở Tuần 2, nhóm tiến hành tái cấu trúc mã nguồn sang kiến trúc Ma trận 2D hướng đối tượng (lớp AES) theo chuẩn đặc tả Rijndael nhằm tăng tính mô-đun hóa và khả năng tích hợp hệ thống. Nội dung trọng tâm của Tuần 3 bao gồm: hoàn thiện mô-đun giải mã đơn khối, xây dựng cơ chế sinh khóa ngẫu nhiên an toàn (CSPRNG) và dẫn xuất khóa PBKDF2 từ mật khẩu, cài đặt cơ chế đệm dữ liệu chuẩn PKCS#7 cùng chế độ hoạt động xích khối CBC. Trên cơ sở đó, nhóm mở rộng thành công khả năng mã hóa và giải mã luồng dữ liệu nhị phân cho các tệp tin đa định dạng và tệp nén ZIP, đảm bảo tính toàn vẹn dữ liệu thông qua kiểm chứng mã băm SHA-256. Quá trình biến đổi ma trận trạng thái State qua từng bước của vòng lặp được tổng hợp trên cùng một bảng ngang nhằm thuận tiện cho việc đối chiếu trung gian.

Từ khóa: AES-128, Ma trận 2D, PKCS#7 Padding, CBC Mode, PBKDF2, Mã hóa File ZIP, SHA-256.

Bảng 1. Đối chiếu tiến độ kỹ thuật giữa Tuần 2 và Tuần 3

Tiêu chí kỹ thuật	Trạng thái Tuần 2	Nâng cấp trong Tuần 3
Kiến trúc mã nguồn	Mảng 1D (bytes) với chỉ số 4*c+r, hướng thủ tục.	Ma trận 2D 4x4 trực quan (class AES) theo chuẩn Rijndael.
Phạm vi xử lý	Chỉ mã hóa đúng 1 khối 16 byte cố định.	Xử lý dữ liệu độ dài bất kỳ nhờ cơ chế đệm PKCS#7.
Chế độ mã hóa	Mã hóa khối đơn (ECB block cipher).	Chế độ khối CBC (Cipher Block Chaining) với IV ngẫu nhiên.
Quản lý khóa	Nhập chuỗi hex 32 ký tự thủ công.	Tự sinh khóa ngẫu nhiên CSPRNG & dẫn xuất PBKDF2 từ mật khẩu.
Ứng dụng thực tế	Chưa hỗ trợ mã hóa file.	Mã hóa & giải mã File / ZIP đa năng, đối chiếu SHA-256 khớp 100%.

Bảng 2. Phân công nhiệm vụ thực hiện trong Tuần 3

Thành viên	Nhiệm vụ cụ thể trong Tuần 3	Đóng góp
Nguyễn Thái Hoàng (106230011)	Cài đặt class AES ma trận 2D, xây dựng PKCS#7 Padding, CBC Mode và mã hóa File ZIP.	50%
Lê Trương Uyển Nhi (106230054)	Cài đặt dẫn xuất khóa PBKDF2, bộ kiểm thử unit test 11 ca test và soạn thảo báo cáo.	50%

1. Giới thiệu và mục tiêu Tuần 3

Trong Tuần 2, nhóm đã nghiên cứu các phép biến đổi toán học cốt lõi của AES-128 theo FIPS 197 trên một khối 16 byte đơn lẻ. Tuy nhiên, để tiến tới việc ứng dụng mã hóa truyền tin trên giao thức TCP Socket ở Tuần 4, hệ thống cần giải quyết ba bài toán thực tế:

1. Dữ liệu thực tế có kích thước ngẫu nhiên: Cần cơ chế đệm (Padding) để chuẩn hóa kích thước dữ liệu về bội số của 16 byte.

2. Tính an toàn khi mã hóa nhiều khối: Cần chế độ hoạt động (Mode of Operation) như CBC nhằm triệt tiêu tính lặp lại của dữ liệu.

3. Tái cấu trúc mã nguồn theo hướng đối tượng (OOP) và ma trận 2D: Đóng gói thuật toán thành lớp AES độc lập với biểu diễn ma trận 2D 4x4 (s[col][row]), giúp tách biệt tầng lõi mật mã với tầng ứng dụng xử lý tệp và truyền thông mạng ở các giai đoạn sau.

2. Kiến trúc giải thuật AES-128 ma trận 2D và luồng State ngang

Khác với cách cài đặt tuần trước, State được biểu diễn dưới dạng danh sách 2 chiều 4 cột, mỗi cột 4 phần tử: s = [[c0r0, c0r1, c0r2, c0r3], ...]. Phép chuyển đổi hai chiều giữa mảng byte phẳng và ma trận 2D được thực hiện qua hai hàm phụ trợ:

```
def bytes2matrix(text: bytes) -> list[list[int]]:
    return [list(text[i:i + 4]) for i in range(0, len(text), 4)]

def matrix2bytes(matrix: list[list[int]]) -> bytes:
    return bytes(sum(matrix, []))
```

Nhờ biểu diễn ma trận 2D, các phép biến đổi được mô tả tường minh và bám sát lý thuyết chuẩn Rijndael:

• SubBytes / InvSubBytes: Duyệt qua từng ô s[i][j] và tra trực tiếp giá trị trong bảng S-box hoặc Inv S-box.

• ShiftRows: Dịch vòng trái các hàng bằng phép gán đồng thời các ô: Hàng 0 giữ nguyên; Hàng 1 dịch 1 (s[0][1], s[1][1], s[2][1], s[3][1] = s[1][1], s[2][1], s[3][1], s[0][1]); Hàng 2 dịch 2; Hàng 3 dịch 3.

• MixColumns: Áp dụng công thức đại số Rijndael rút gọn qua phép xtime(a) = ((a << 1) ^ 0x1B) & 0xFF nếu bit cao bằng 1. InvMixColumns cũng được tối ưu hóa theo sách The Design of Rijndael mà không cần nhân ma trận 4x4 phức tạp.

2.1 Biểu diễn trạng thái State qua các phép biến đổi của Vòng 1

Nhằm theo dõi sự thay đổi dữ liệu liên tục qua từng phép biến đổi trung gian của giải thuật, Bảng 3 biểu diễn chuỗi trạng thái State của Vòng 1 trên cùng một hàng ngang:

1. State đầu vào Vòng 1	2. Sau SubBytes	3. Sau ShiftRows	4. Sau MixColumns	5. Kết quả sau AddRoundKey

1. State đầu vào Vòng 1	2. Sau SubBytes	3. Sau ShiftRows	4. Sau MixColumns	5. Kết quả sau AddRoundKey
00 40 80 c0 10 50 90 d0 20 60 a0 e0 30 70 b0 f0	63 09 cd ba ca 53 60 70 b7 d0 e0 e1 04 51 e7 8c	63 09 cd ba 53 60 70 ca e0 e1 b7 d0 8c 04 51 e7	5f 57 f7 1d 72 f5 be b9 64 bc 3b f9 15 92 29 1a	89 85 2d cb d8 5a 18 12 10 ce 43 8f e8 68 d8 e4

Nhận xét: Bảng luồng ngang trên minh chứng trực quan: SubBytes thay đổi giá trị từng ô nhưng giữ nguyên tọa độ; ShiftRows đảo vị trí byte giữa các cột; MixColumns khuếch tán dữ liệu trong từng cột; và AddRoundKey đưa khóa bí mật K1 vào State.

3. Phương pháp thực hiện các mô-đun mở rộng

3.1 Cơ chế tự động tạo khóa và dẫn xuất khóa (Key Derivation)

Trong các ứng dụng thực tế, người dùng không thể tự nhớ hoặc gõ chuỗi hex 32 ký tự. Nhóm đã triển khai hai phương thức sinh khóa tự động:

1. Sinh khóa ngẫu nhiên an toàn: Sử dụng os.urandom(16) từ bộ sinh số ngẫu nhiên an toàn của hệ điều hành (CSPRNG), đảm bảo khóa có entropy cực đại và không thể đoán trước.

2. Dẫn xuất khóa từ mật khẩu (PBKDF2): Khi người dùng nhập một mật khẩu bất kỳ (ví dụ: 'MatKhau@123'), hệ thống kết hợp với một chuỗi muối ngẫu nhiên (Salt 16 byte) và chạy 100.000 vòng băm qua thuật toán PBKDF2-HMAC-SHA256 để tạo ra đúng 16 byte khóa AES-128. Cơ chế này chống lại tấn công vét cạn (brute-force) và bảng băm cầu vồng (rainbow tables).

3.2 Cơ chế đệm dữ liệu chuẩn PKCS#7 (Padding & Unpadding)

Thuật toán mã hóa khối AES bắt buộc dữ liệu đầu vào phải là bội số của 16 byte. Nhóm cài đặt cơ chế đệm chuẩn PKCS#7 (RFC 5652):

Gọi L là độ dài dữ liệu gốc tính theo byte. Số lượng byte cần đệm thêm P_len ($1 \leq P_len \leq 16$) được xác định theo công thức:

$$P_len = 16 - (L \bmod 16)$$

• Quy tắc thêm đệm: Chèn thêm đúng P_len byte, mỗi byte đều mang giá trị đúng bằng P_len. Trường hợp đặc biệt khi dữ liệu gốc đã chia hết cho 16 byte (L mod 16 = 0), hệ thống vẫn chèn thêm đủ một khối đệm 16 byte mang giá trị 0x10. Quy ước này giúp bên giải mã phân biệt chính xác dữ liệu gốc với byte đệm mà không làm cụt đuôi dữ liệu nhị phân.

• Quy tắc gỡ đệm: Đọc giá trị byte cuối cùng P_len, kiểm tra xem đủ P_len byte cuối có cùng giá trị P_len hay không. Nếu kiểm tra thất bại, hệ thống phát sinh ngoại lệ ValueError để cảnh báo dữ liệu bị giải mã sai hoặc bị tấn công.

```
def pad(data: bytes, block_size: int = 16) -> bytes:
    padding_len = block_size - (len(data) % block_size)
    return data + bytes([padding_len] * padding_len)

def unpad(data: bytes, block_size: int = 16) -> bytes:
    padding_len = data[-1]
    if padding_len < 1 or padding_len > block_size or data[-padding_len:]
!= bytes([padding_len] * padding_len):
        raise ValueError('Padding PKCS#7 bị hỏng hoặc sai giá trị kiểm
tra')
    return data[:-padding_len]
```

3.3 Chế độ hoạt động chuỗi khối CBC (Cipher Block Chaining)

Nếu sử dụng chế độ ECB (mã hóa từng khối độc lập), các khối bản rõ giống nhau sẽ tạo ra khối bản mã giống nhau, làm lộ hình dạng dữ liệu (kinh điển như hình ảnh chim cánh cụt ECB Penguin). Chế độ CBC giải quyết triệt để vấn đề này bằng cơ chế ghép xích:

• Mã hóa CBC: Khối bản rõ đầu tiên P0 được XOR với Vector khởi tạo ngẫu nhiên (IV 16 byte) rồi mới mã hóa: C0 = E_k(P0 ⊕ IV). Các khối tiếp theo được XOR với khối mã hóa đứng trước: C_i = E_k(P_i ⊕ C_{i-1}).

• Giải mã CBC: Thực hiện ngược lại: P0 = D_k(C0) ⊕ IV và P_i = D_k(C_i) ⊕ C_{i-1}.

Nhờ có IV ngẫu nhiên sinh từ os.urandom(16) cho mỗi lần mã hóa, cùng một file văn bản khi mã hóa hai lần sẽ cho ra hai bản mã hoàn toàn khác nhau, đảm bảo tính an toàn ngữ nghĩa (Semantic Security) và triệt tiêu hoàn toàn hiện tượng lộ mẫu dữ liệu.

3.4 Cài đặt mô-đun mã hóa và giải mã File / ZIP đa năng

Trong hệ điều hành, tất cả các tệp tin (từ văn bản .txt, tài liệu .pdf đến tệp nén .zip) đều được biểu diễn dưới dạng luồng byte nhị phân. Do đó, nhóm xây dựng mô-đun mã hóa file tổng quát:

1. Đọc tệp gốc dưới dạng binary (rb) và tính mã băm SHA-256 ban đầu.

2. Áp dụng đệm PKCS#7 và mã hóa toàn bộ dữ liệu bằng chế độ CBC.

3. Ghi file đầu ra theo cấu trúc chuẩn: [16 bytes IV] + [Ciphertext CBC] (hoặc nhúng Salt đối với phương thức dùng mật khẩu).

4. Khi giải mã: Tách IV, giải mã phần ciphertext, gỡ đệm PKCS#7, ghi lại tệp gốc và đối chiếu mã băm SHA-256.

4. Thực nghiệm và kết quả kiểm thử

4.1 Kiểm thử tự động với bộ test NIST và tính năng mới

Toàn bộ mã nguồn Tuần 3 được kiểm thử nghiêm ngặt thông qua thư viện unittest trong file test_aes.py với 11 test method độc lập:

Hạng mục kiểm thử	Nội dung & Dữ liệu kiểm tra	Kết quả
Độ chuẩn xác FIPS 197	Vector C.1 (Ciphertext & Plaintext) và 4 khối NIST Core.	5/5 vector PASS
Cơ chế đệm PKCS#7	Kiểm thử các độ dài biên: 0, 1, 15, 16, 17, 32 byte và từ chối đệm sai.	PASS (10/10 ca)
Dẫn xuất khóa PBKDF2	Tạo khóa 16 byte từ mật khẩu người dùng với muối ngẫu nhiên.	PASS
Mã hóa chế độ CBC	Mã hóa chuỗi văn bản UTF-8 và luồng nhị phân ngẫu nhiên đa khối.	PASS
Mã hóa File ZIP đa năng	Nén tài liệu thành file ZIP, mã hóa CBC, giải mã đối chiếu SHA-256.	PASS (Khớp 100%)

4.2 Thực nghiệm mã hóa và giải mã file nén ZIP thực tế

Để chứng minh tính thực tiễn cao của mô-đun, nhóm đã thực hiện kịch bản demo tự động (lệnh: python demo.py --demo-zip) với một tệp nén sample_archive.zip chứa 2 file dữ liệu (baocao_pbl3.txt và data/students.csv). Kết quả đo lường thực nghiệm ghi nhận:

```
KẾT QUẢ THỰC NGHIỆM MÃ HÓA FILE ZIP (demo.py --demo-zip):
1. File ZIP gốc: sample_archive.zip (379 bytes)
   -> Mã băm SHA-256 gốc :
13b559cdc000e82588b28df587431d2114c12d10ea8c4965123707a7f9aa47b6
2. File sau mã hóa AES-128-CBC: sample_archive.zip.enc (400 bytes = 16B IV
+ 384B Ciphertext)
3. File sau khi giải mã: sample_archive_recovered.zip (379 bytes)
   -> Mã băm SHA-256 sau giải mã:
13b559cdc000e82588b28df587431d2114c12d10ea8c4965123707a7f9aa47b6
4. Đối chiếu mã băm toàn vẹn SHA-256: PASS (Trùng khớp 100% từng bit)
5. Mở và giải nén file ZIP: Đọc chính xác toàn bộ 2 file tài liệu bên trong
không có lỗi!
```

Kết quả trên khẳng định: Hệ thống không những bảo vệ an toàn nội dung tệp tin mà còn bảo toàn nguyên vẹn 100% cấu trúc định dạng nén phức tạp của file ZIP.

5. Kết luận và kế hoạch Tuần 4

Trong Tuần 3, nhóm đã hoàn thành toàn diện các mục tiêu đề ra:

• Tái cấu trúc thành công giải thuật AES-128 sang mô hình Ma trận 2D và Hướng đối tượng (lớp AES), giúp mã nguồn tường minh, đạt tính đóng gói cao và thuận tiện cho việc mở rộng truyền thông Socket ở tuần tiếp theo.

• Cài đặt hoàn chỉnh cơ chế tạo khóa tự động (CSPRNG & PBKDF2), đệm dữ liệu PKCS#7 và chế độ xích khối CBC.

• Hiện thực hóa thành công việc mã hóa/giải mã file thực tế và file ZIP đa năng, có kiểm chứng mã băm toàn vẹn SHA-256 đạt chuẩn.

Kế hoạch Tuần 4: Sử dụng toàn bộ module AES-CBC đã hoàn thiện trong Tuần 3 để tích hợp vào ứng dụng mạng Client - Server truyền nhận file an toàn qua giao thức TCP Socket.

Tài liệu tham khảo

[1] National Institute of Standards and Technology (NIST), Advanced Encryption Standard (AES), FIPS PUB 197, 2001 (Updated 2023).

[2] Joan Daemen, Vincent Rijmen, The Design of Rijndael: AES - The Advanced Encryption Standard, Springer, 2002.

[3] Lucas Boppre, Pure Python AES implementation, GitHub Repository: boppreh/aes, 2020.

[4] RFC 5652, Cryptographic Message Syntax (CMS) - Section 6.3: Content-encryption process (PKCS#7 Padding).

[5] NIST Special Publication 800-38A, Recommendation for Block Cipher Modes of Operation: Methods and Techniques, 2001.